Name: Atiksh Borundia
Reg. No.: 24011101009

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory
Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2 Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3 Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4 Dataset

Dataset: Fashion-MNIST

Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5 Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6 Experimental Procedure

6.1 Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Dataset dimensions obtained: Training data shape (60000, 28, 28); Training labels shape (60000,); Testing data shape (10000, 28, 28); Testing labels shape (10000,).

6.2 Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Tensor shapes before preprocessing: Training (60000, 28, 28), Testing (10000, 28, 28). Tensor shapes after preprocessing: Training (60000, 784), Testing (10000, 784); one-hot encoded labels: Training (60000, 10), Testing (10000, 10).

6.3 Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Layer (type)	**Output Shape**	**Param #**
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB) Trainable params: 109,386 (427.29 KB) Non-trainable params: 0

6.4 Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Training accuracy increasred from 0.8210 (epoch 1) to 0.9343 (epoch 20); training loss decreased from 0.5072 to 0.1722; validation accuracy increased from 0.8298 (epoch 1) to 0.8879 (epoch 20). Total baseline training time was approximately 117 seconds across the 20 logged epochs.

6.5 Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Baseline test performance: Accuracy 0.8787, Precision 0.8777, Recall 0.8787, F1-score 0.8772 (weighted average).

7 Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

RandomizedSearchCV was used (`scikit-learn 1.5.2`, `scikeras 0.13.0`), with `n_iter=5` and `cv=5` (5-fold cross-validation), fitted on the full training set ("5 iterations for 5 folds of Cross Validation, equalling 25 fits").

8 Mandatory Plots

Figure 1: Sample Images

Inference: The dataset contains 10 classes of clothing categories which look very similar visually, hence dense layers may be required and errors may be observed more so.

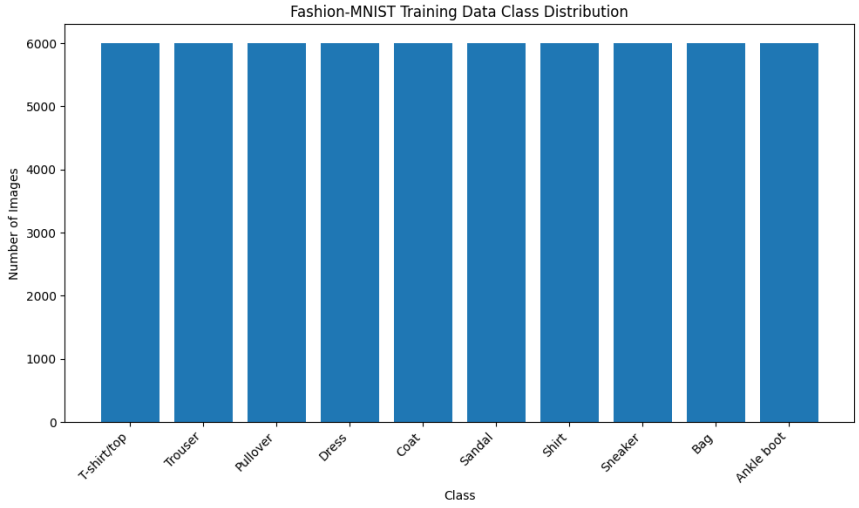


Figure 2: Class Distribution

Inference: The dataset is balanced, hence we can say that accuracy may be a good measure as there maybe no bias of more number of classification done towards dominant classes.

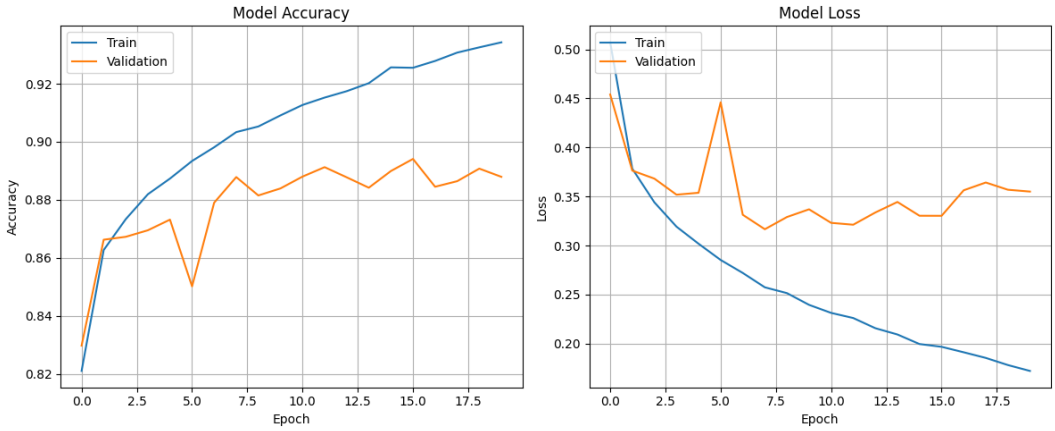


Figure 3: Training Accuracy vs Epoch (left) and Training Loss vs Epoch (right), with Validation curves overlaid

Inference (Training Accuracy vs Epoch): The training accuracy rises consistently across all epochs with a very good and reasonable accuracy fitting the training dataset.

Inference (Validation Accuracy vs Epoch): Validation accuracy rises with a few sharp increases and decreases showing randomization but it stabilizes after a point indication early signs of overfitting.

Inference (Training Loss vs Epoch): The Training loss doesn't increase at any epoch finally converging with the help of Adam Optimizer.

Inference (Validation Loss vs Epoch): The sharp peaks in validation loss confirm the signs of overfitting observed earlier.

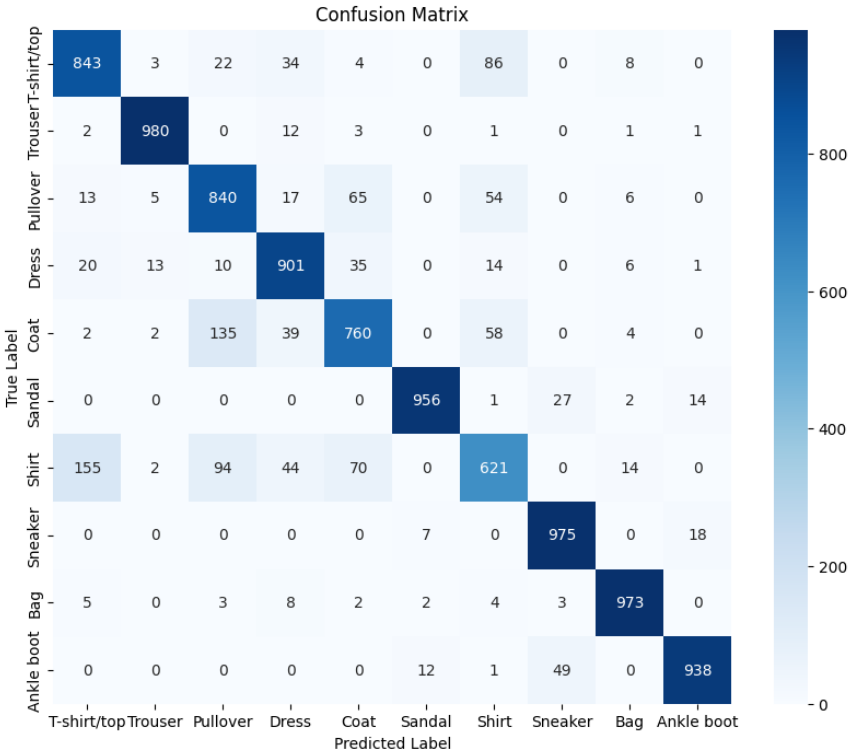


Figure 4: Confusion Matrix (Baseline Model)

Inference: Most predictions are right at about 98 percent approximately, but the Confusion Matrix shows that shirts and trouser and pullovers and coats were confused which was expected due to visual similarity. Shirts have the lowest recall of all the classes.

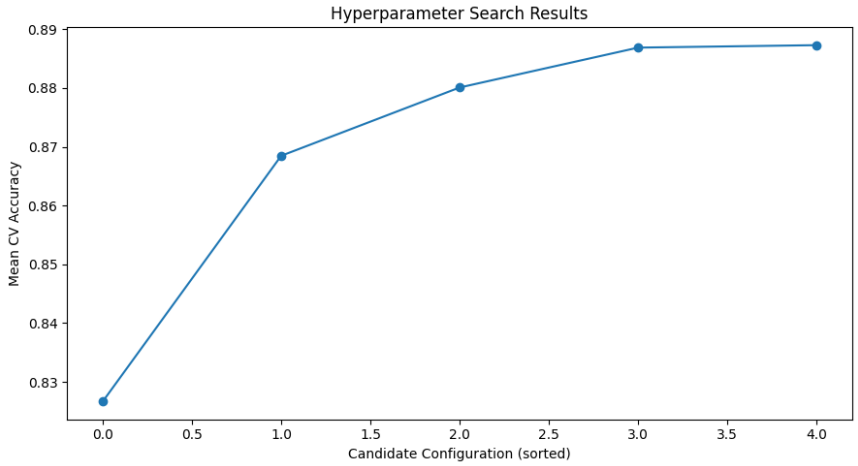


Figure 5: Hyperparameter Search Results (5 randomly sampled configurations, 5-fold CV)

Inference: Across the 5 random samples, the mean CV accuracy increased from about 0.82 to 0.887. It rises and flattens but since the number of iterations were very small due to processing time limits, the resultant accuracy that is achieved is pretty good which might increase even more if the iterations would have been higher.

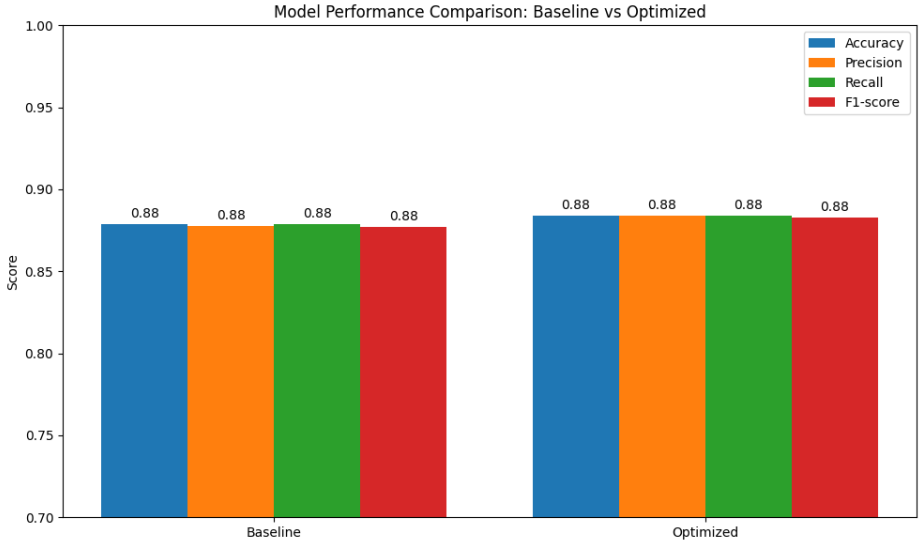


Figure 6: Best Model Accuracy Comparison

Inference: The onlyl marginal increase in scores can be attributed to the very low iterations used, which are less than even 1% of the total search space given in the experiment.

9 Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	Adam
Activation Function	Tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8873
Testing Accuracy	0.8841

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8787	0.8841
Precision	0.8777	0.8838
Recall	0.8787	0.8841
F1-score	0.8772	0.8828
Training Time	≈117 s (20 epochs)	53.5 s (20 epochs)

Inference: The optimized model's Accuracy (0.8841), Precision (0.8838), Recall (0.8841), and F1-score (0.8828) on the test set are all higher than the baseline model's scores (0.8787, 0.8777, 0.8787, 0.8772), showing that it did better than manually putting hyperparameters for the model. This indicates that if the iterations would have been higher, the difference could have been significantly higher.

Baseline Classification Report

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.81	0.84	0.83	1000
Trouser	0.98	0.98	0.98	1000
Pullover	0.76	0.84	0.80	1000
Dress	0.85	0.90	0.88	1000
Coat	0.81	0.76	0.78	1000
Sandal	0.98	0.96	0.97	1000
Shirt	0.74	0.62	0.68	1000
Sneaker	0.93	0.97	0.95	1000
Bag	0.96	0.97	0.97	1000
Ankle boot	0.97	0.94	0.95	1000
Accuracy		0.88		10000
Macro avg	0.88	0.88	0.88	10000
Weighted avg	0.88	0.88	0.88	10000

Optimized Classification Report

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.81	0.87	0.84	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.77	0.84	0.80	1000
Dress	0.89	0.90	0.89	1000
Coat	0.80	0.81	0.80	1000
Sandal	0.97	0.95	0.96	1000
Shirt	0.76	0.62	0.68	1000
Sneaker	0.93	0.97	0.95	1000
Bag	0.96	0.97	0.97	1000
Ankle boot	0.97	0.95	0.96	1000
Accuracy		0.88		10000
Macro avg	0.88	0.88	0.88	10000
Weighted avg	0.88	0.88	0.88	10000

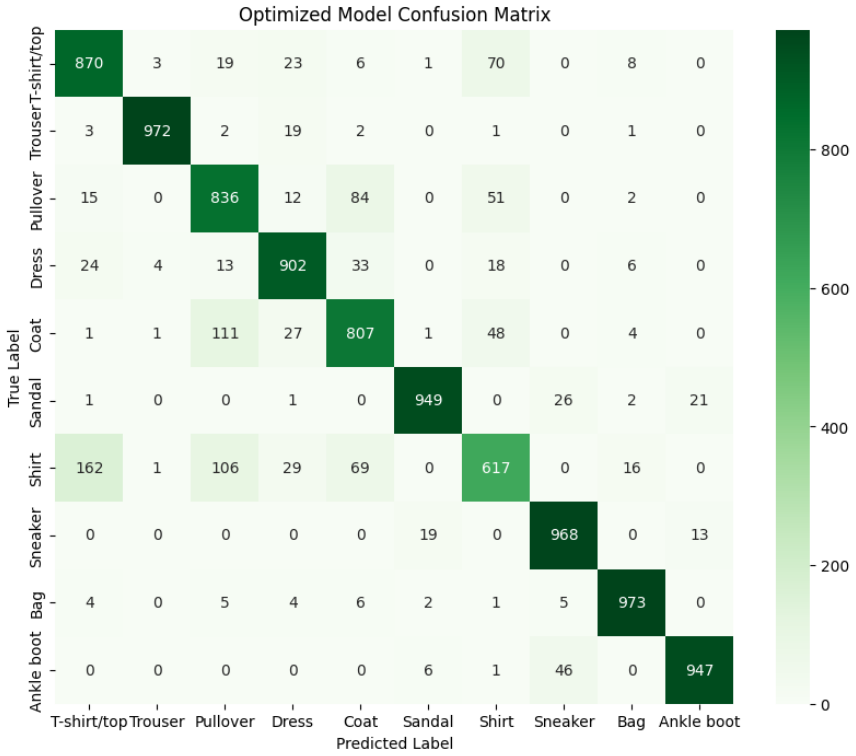
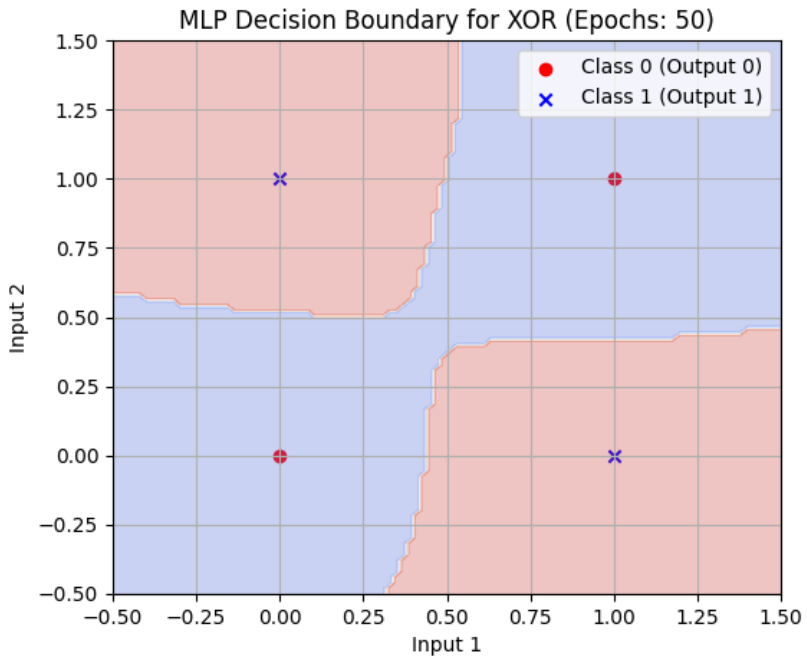


Figure 7: Confusion Matrix (Optimized Model)

Inference: As the iterations were less, the results look similar to original confusion matrix, with few outputs producing lesser results and few producing more like Coat and Ankle Boot.

10 XOR Gate using MLP

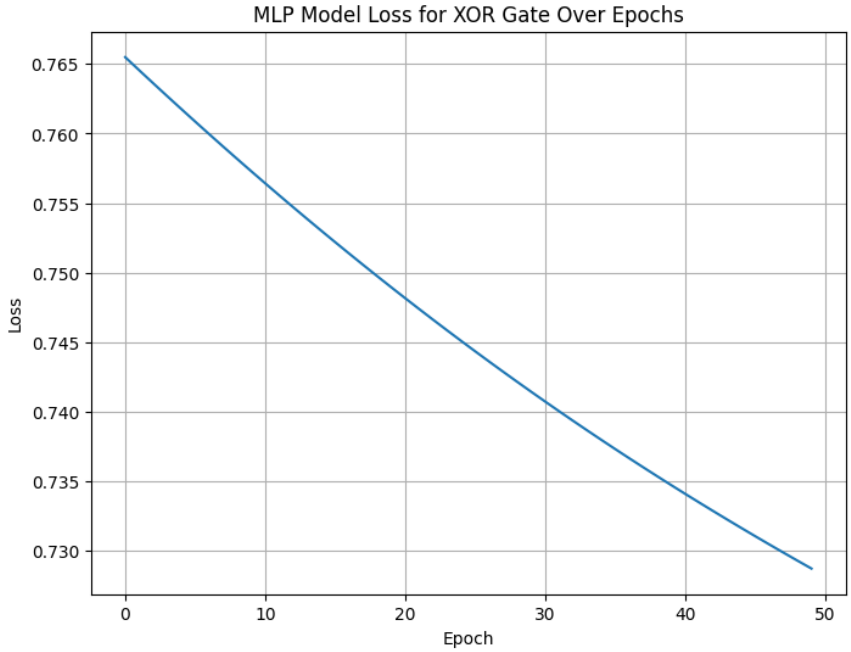


The XOR GATE has two points of one class between the other two points. Both of them intrude each other's space as seen in the above plot. They are non-linearly classifiable in the current 2D space. Hence, XOR cannot be implemented by a Simple Perceptron alone. Here is the architecture of the MLP:

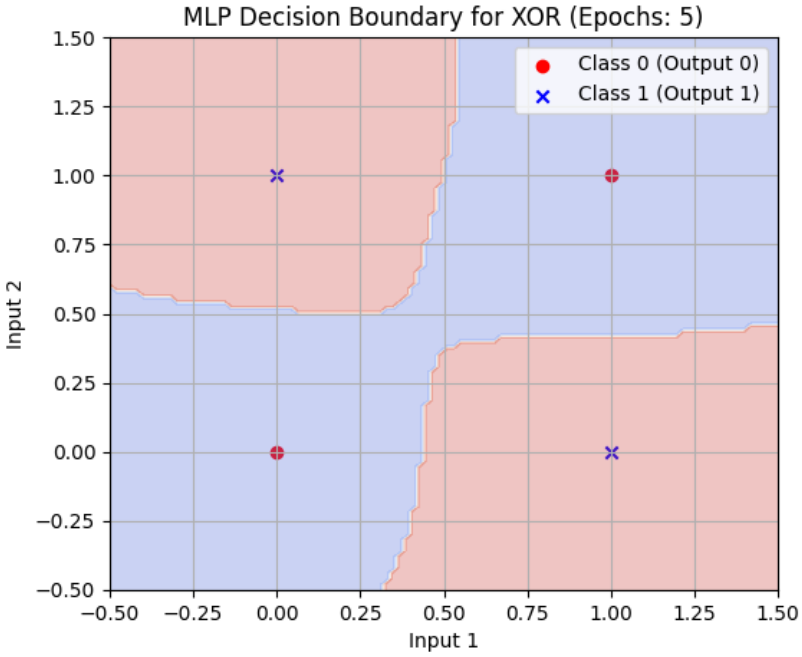
Table 1: Model Architecture

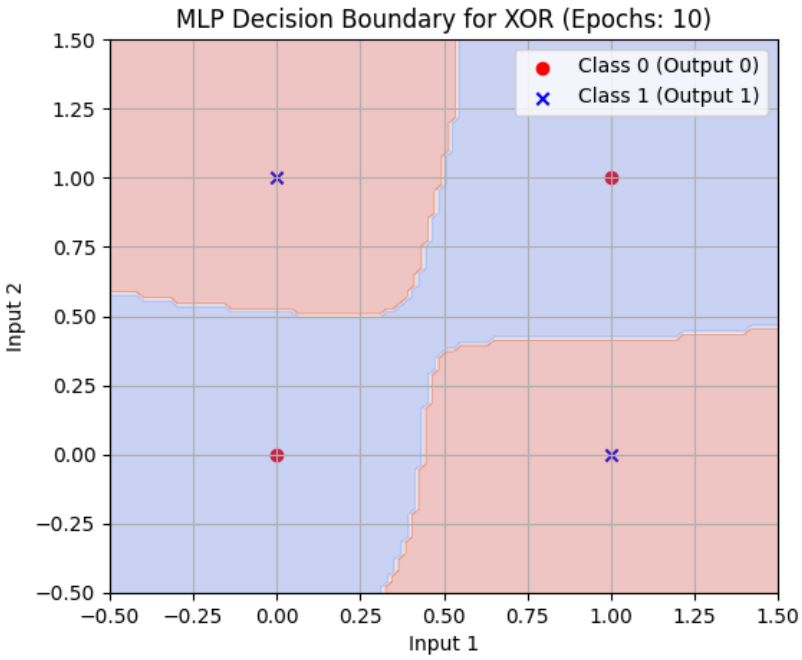
Layer (Type)	**Output Shape**	**Parameters**
Input Layer	(None, 2)	0
Dense (4 neurons, ReLU)	(None, 4)	12
Dense (1 neuron, Sigmoid)	(None, 1)	5
Total Parameters		**17**
Trainable Parameters		17
Non-trainable Parameters		0

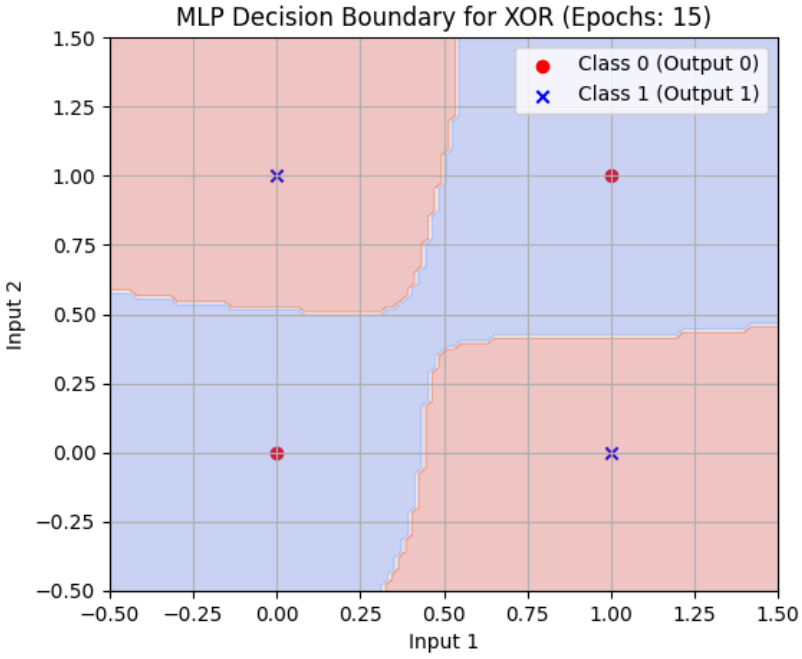
It took around 50 epochs and a tanh function in the hidden layer and sigmoid layer in the output layer to introduce the non linearity required to classify this and to decrease the model loss even though the points were classified. The output was showing 100% accuracy at the starting of training only but the model loss took around 50 epochs to the Model Loss as sigmoid function says the probability of the prediction, not exactly predicted the output. The hidden layer ensures that abstract representations were created which could be made linear from the original non linear input of XOR. The Universal Approximation Theorem states that MLP with atleast one hidden layer can approximate any continous function

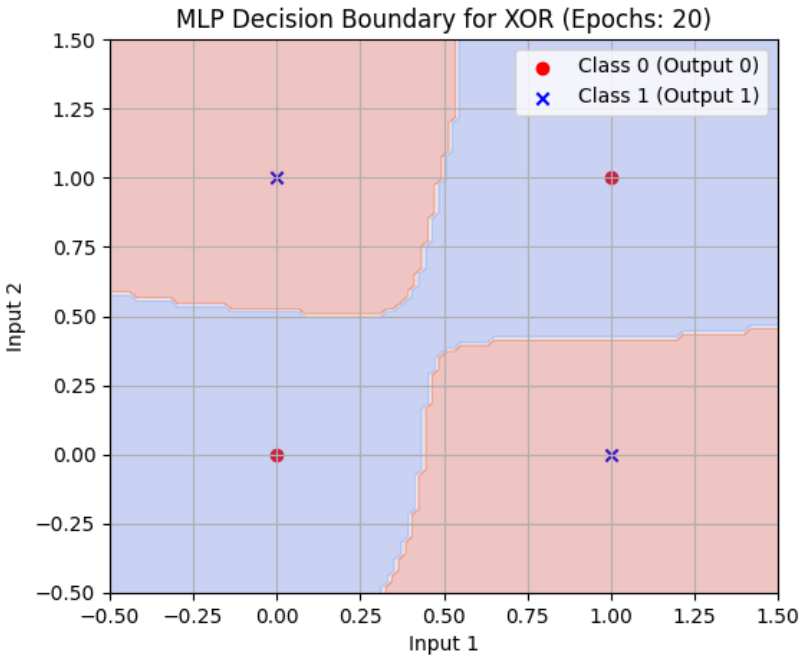

MLP Model Loss for XOR Gate Over Epochs
Loss
Epoch
0.765
0.760
0.755
0.750
0.745
0.740
0.735
0.730
0
10
20
30
40
50

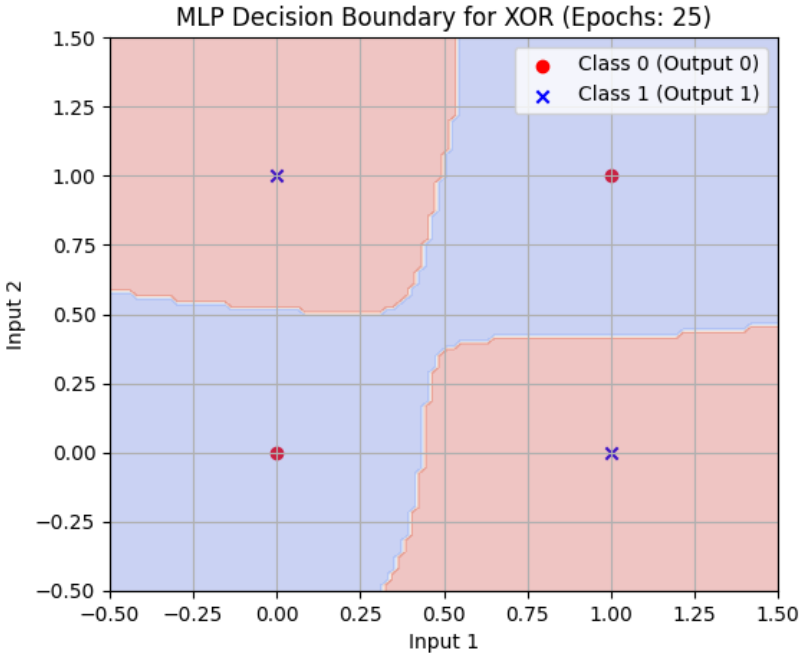
Epochs

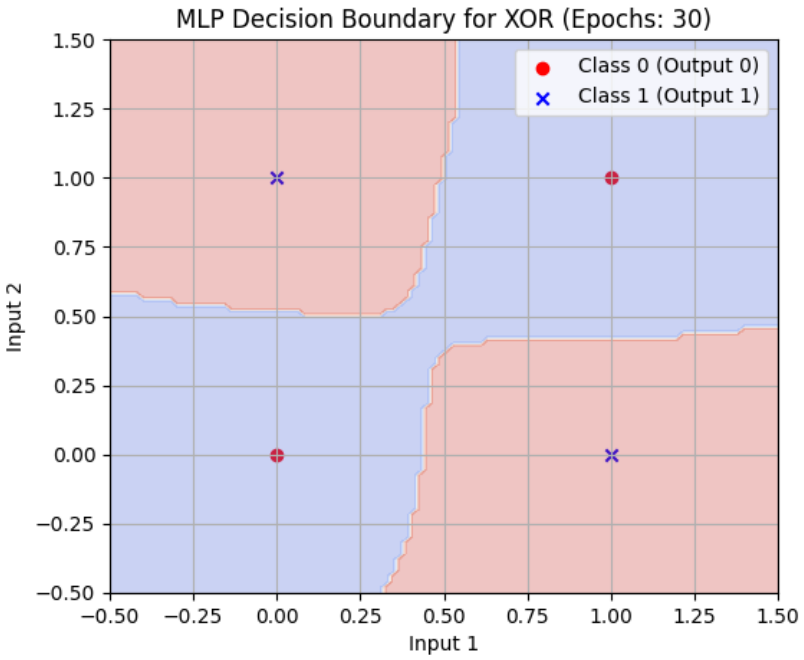

MLP Decision Boundary for XOR (Epochs: 5)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50

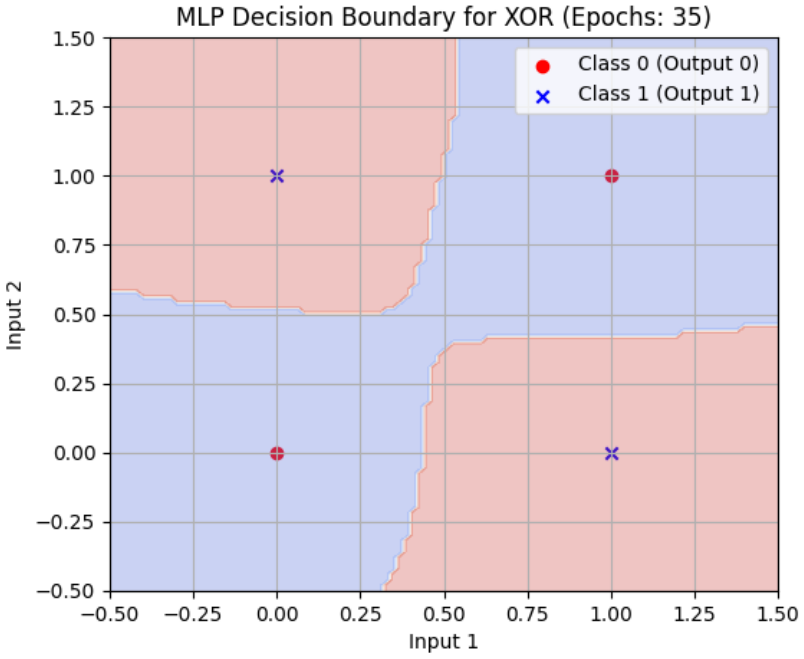

MLP Decision Boundary for XOR (Epochs: 10)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50

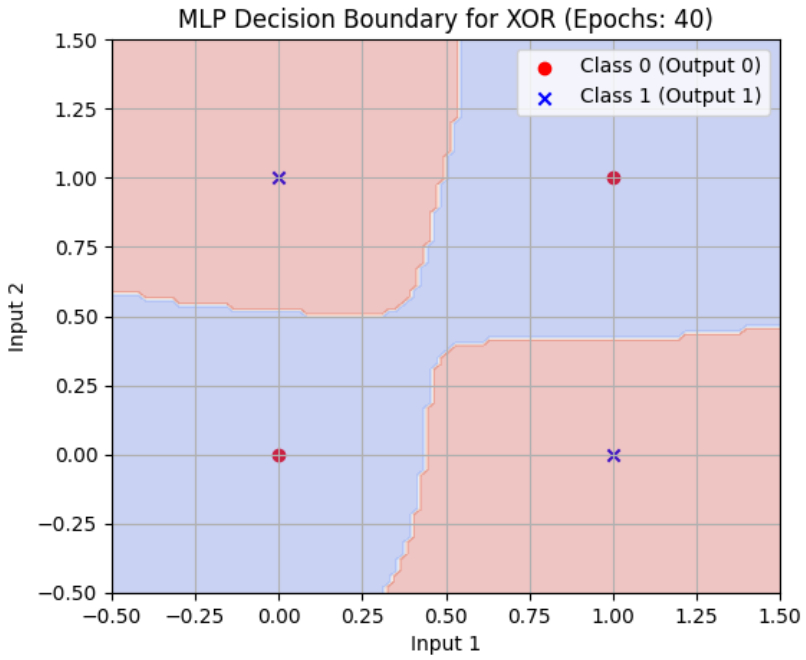

MLP Decision Boundary for XOR (Epochs: 15)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50

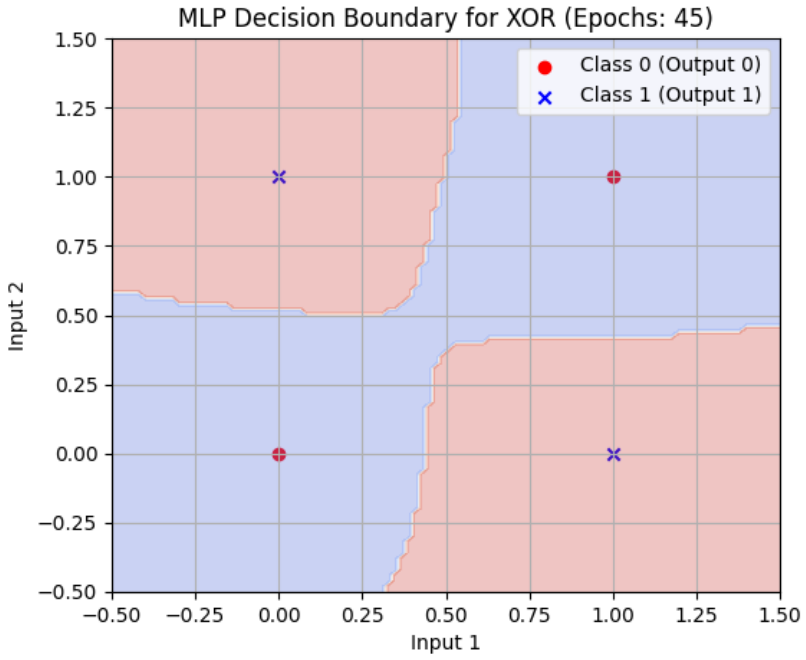

MLP Decision Boundary for XOR (Epochs: 20)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50


MLP Decision Boundary for XOR (Epochs: 25)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50


MLP Decision Boundary for XOR (Epochs: 30)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50


MLP Decision Boundary for XOR (Epochs: 35)
Class 0 (Output 0)
Class 1 (Output 1)
Input 1
Input 2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50





11 Discussion

Answer the following:

1. Which optimization method was used?
   Randomized Search Cross-Validation was used, via SciKeras's `KerasClassifier` wrapper combined with scikit-learn's `RandomizedSearchCV`, fitted on the full training set with 25 total model fits (5

candidates × 5 folds).

2. Which hyperparameters were selected?
   Hidden Layers: 1; Hidden Neurons: 256; Learning Rate: 0.001; Batch Size: 64; Optimizer: Adam; Activation Function: Tanh; Epochs: 20; Dropout Rate: 0.2. Mean CV accuracy : `0.8873`.
3. Did optimization improve performance?
   Yes. It increased performance over all the four main metrics like accuracy, F1, precision, etc. As selecting randomly from a random search space designed to increase the potential chances of selecting better hyperparameters is better for the model before it starts training.
4. Which hyperparameter had the greatest impact?
   Isolating a single hyperparameter for this small experiment with only 5 iterations would be difficult and biased.
5. Compare Grid Search and Randomized Search.
   Grid Search uses brute force technique of finding the best optimum given it searches whole search space for all combinations finding the best metrics scored out of all of the combinations available.**Randomized Search** instead based on the given number of iterations finds randomly the combination and does this till it reaches the number of iterations given. There is no guarantee of finding an optimum or a good result but it is computationally cheaper than the former.
6. Recommend the best MLP configuration.
   Based on this experiment, the recommended MLP configuration is: 1 hidden layer of 256 neurons with Tanh activation, Adam optimizer at learning rate 0.001, dropout rate 0.2, trained for 20 epochs with batch size 64. This configuration got a CV accuracy of 0.8873 and a testing accuracy of 0.8841, outperforming the manually chosen baseline (784 → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax), 20 epochs, batch size 32, testing accuracy 0.8787) on every evaluation metric.

12 Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

Source Code

`https://colab.research.google.com/drive/10Gnz6Fw-yrOaWbNzOHXerk0WDDIz5tHT`

Conclusion

So to conclude, this experiment implemented a Multi-Layer Perceptron for multi-class image classification on the Fashion-MNIST dataset using TensorFlow/Keras. A baseline model was built using the given parameters and architecture and the results were recorder. Then, the Hyperparameter Optimization technique of Random CV was used to find better hyperparameters from the given search space and it gave better results. For bigger models and experiments, the hyperparameters' selection is very crucial for the model to perform better and be computationally cheaper, as manual selection may cause a lot of compute requires and waste time switching to another parameter based on intuition and guesstimate.

13 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.

5. TensorFlow/Keras Documentation.